

주요코드설명

Player class

```
48
49 //각 캐릭터마다 다룰트 무기나 필살기가 존재하는 캐릭터가 있으므로 구현이 다 달라서 abstract로 설정해놓고
50 //각 캐릭터에서 직접 생략
51 public abstract void settingWeapons(ArrayList<Weapon> garrage, int num);
52
53 public abstract void attack(Player p);
54
55 public abstract void attack_choose(Player p, Weapon w);
56
57 public Weapon choose_weapon() {
58     int a = this.weapons.size();
59     int rand = (int)((Math.random()*10))%a;
60     return this.weapons.get(rand);
61 }
62
```

1. 무기창고에서 받아온 무기들을 통해 내 무기 창고를 채우는 메소드,
2. 공격을 위한 메소드(두 가지가 있는 것은 무기가 랜덤하게 골라져서 공격하는 방식과 직접 무기를 선택해서 공격하는 방식이 있기 때문)
3. 공격하기 버튼을 통해 공격할 때는 있는 무기중에 랜덤하게 골라서 공격을 하므로 그 때 필요한 메소드

1,2는 자식 클래스인 캐릭터마다 가질 수 있는 무기와 사용방법이 다르므로 추상 클래스로 구현함.

Player class의 자식 클래스 중 한 캐릭터인 토르 class

```
public class 토르 extends Player implements Hittable, Useable{
    int 망치쓰기 = 90;
    int 칼다루기 =10;
    주먹 r = new 주먹("주먹",60);
    망치 h = new 망치("망치_토르 스페셜",140);
    Weapon 솔 = new Weapon("솔",-10);
    Weapon 필살기 = new Weapon("필살기",150);
    //토르의 특별한 공격 메소드 두 가지를 실현하기 위한 무기도 넣어둔다.

    public 토르(String name) {
        this.name = name;
        this.setHp(500);
        this.setPower(60);
        this.setimgfile("C:\\Users\\kirin\\Desktop\\\\uAC1D\\uC9DA\\uC774\\uB8F8\\uC9C0\\uADF8\\uB9BC01.jpg");
    }
}
```

1. 사용 무기에 해당하는 추가 능력치와 가지고 태어나는 기본 무기들을 정의해준다.
2. 생성자에서는 이름, Hp, Power, 그리고 사진파일 경로를 설정한다.

```
public void 솔마시기(){
    //솔마시면 자신의 HP가 멀어지게 구현하기
    this.setHp(this.getHp()+솔.getPower());
}

public void 때리기(Player p) {
    p.setHp(p.getHp()-this.getPower());
}

public void 사용하기(망치 w, Player p) {
    p.setHp(p.getHp()-망치쓰기);
}

public void 사용하기(칼 w, Player p) {
    p.setHp(p.getHp()-w.getPower()-this.칼다루기);
}

public void 사용하기(Weapon t, Player p) {
    if(t.name == "필살기") p.setHp(p.getHp()-t.getPower());
}
}
```

각 무기를 사용하는 방법에 따라 정의된 공격 메소드

```

public void attack(Player p) {
    cur_weapon = choose_weapon();

    System.out.println(this.name+": "+cur_weapon.name+"(으)로 "+p.name+" 공격!");

    if(cur_weapon instanceof 망치) {
        this.사용하기(cur_weapon, p);
    }
    if(cur_weapon instanceof 주먹) {
        this.때리기(p);
    }
    if(cur_weapon instanceof 칼) {
        this.사용하기(cur_weapon, p);
    }
    if(cur_weapon.name == "활살기") {
        this.사용하기(cur_weapon, p);
    }

    if(cur_weapon.name == "술") {
        this.술마시기();
        System.out.println("술에 취해 상대를 공격 못하고 자신의 HP가 낮았습니다.");
    }
    this.show_header();
    this.show();
    p.show();
}

```

choose_weapon()을 통해 내가 갖고 있는 무기 중에 임의로 선택돼서 공격을 할 때 사용되는 attack 메소드

```

@Override
public void attack_choose(Player p, Weapon w) {
    System.out.println(this.name+": "+w.name+"(으)로 "+p.name+" 공격!");

    if(w instanceof 망치) {
        this.사용하기(w, p);
    }
    if(w instanceof 주먹) {
        this.때리기(p);
    }
    if(w instanceof 칼) {
        this.사용하기(w, p);
    }
    if(w.name == "활살기") {
        this.사용하기(w, p);
    }

    if(w.name == "술") {
        this.술마시기();
        System.out.println("술에 취해 상대를 공격 못하고 자신의 HP가 낮았습니다.");
    }
    this.show_header();
    this.show();
    p.show();
}

```

사용자가 무기를 직접 선정하여 공격하는 메소드

```

public void settingWeapons(ArrayList<Weapon> garrage, int num) {

    //토르의 기본 무기인 망치를 무기창고에 미리 넣어놓는다.

    setWeapons(r);
    setWeapons(h);

    setWeapons(술);
    setWeapons(필살기);

    for(int i =0;i<num;){
        int k = (int)((Math.random()*10)%garrage.size());
        if(garrage.get(k) instanceof 총 || garrage.get(k) instanceof 활 ) continue;
        setWeapons(garrage.get(k));
        i++;
    }
    //그 외의 무기들은 사용자가 지정한 숫자 만큼 추가적으로 공용 무기창고에서 가져온다.
}

```

기본적으로 갖고 태어나는 무기들과, 공용 무기창고를 통해 내 무기 창고를 채우는 메소드

활, 총의 메소드를 보이기 위해 블랙위도우의 메소드 차용

```

public void 쓰기(총 w, Player p) {
    if(w.get총알()<=0) {
        System.out.println(w.name+" : 총알이 다 떨어져 공격하지 못합니다");
        System.out.println(w.name+" : 무기창고에서 제거");
        this.removeWeapons(w);
        this.show_w();
    }
    else {
        p.setHp(p.getHp()-총쏘기-w.getPower());
        w.set총알(w.get총알()-1);
        System.out.println("남은 총알: "+ w.get총알());
    }
}

```

총알/화살이 남아있을 때 총/ 활로 공격하면 총알/화살 하나가 줄어 들고, 콘솔, GUI 화면엔 남은 총알/화살 개수를 띄운다. 총알이 0개이면 해당 무기를 무기창고에서 없애고, 나머지 무기가 들어있는 무기창고 내역을 띄워주고, 그 턴에서는 공격을 못한다.

무기 class

```

public class Weapon {
    public String name;
    private int power;

    public Weapon() {}
    public Weapon(String name) {
        this.name= name;
    };
    public Weapon(String name, int power) {
        this.name = name;
        setPower(power);
    }
    public int getPower() {
        return power;
    }

    public void setPower(int power) {
        this.power = power;
    }
}

```

인자에 따른 몇 개의 생성자 존재.

무기 Class의 자식 클래스 중 하나인 총 Class

```
public class 총 extends Weapon{
    private int 총알;

    public 총() {
        this.setPower(80);
    }

    public 총(String name) {
        super(name, 80);
    }

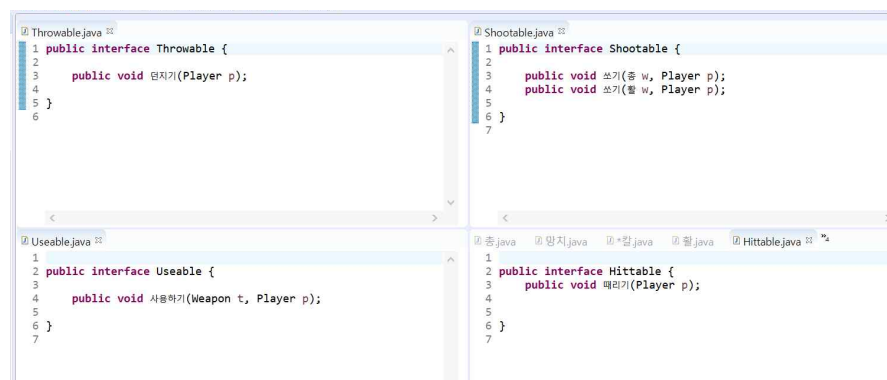
    public 총(String name, int 총알) {
        super(name, 80);
        this.set총알(총알);
    }

    public int get총알() {
        return 총알;
    }

    public void set총알(int 총알) {
        this.총알 = 총알;
    }
}
```

생성자는 무기 class의 생성자를 상속받아 사용함

공격 방식에 맞게 각 플레이어가 implements한 interface들



Game Window

```
public GameWindow() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 953, 702);
    contentPane = new JPanel();
    contentPane.setBackground(Color.WHITE);
    contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane);
    contentPane.setLayout(null);

    panel1.setBackground(Color.WHITE);
    panel1.setBounds(12, 10, 292, 632);
    contentPane.add(panel1);
    panel1.setLayout(null);

    p1공격버튼.setFont(new Font("a옛날사진관4", Font.PLAIN, 16));
    p1공격버튼.setBounds(12, 439, 93, 47);
    panel1.add(p1공격버튼);

    p2공격버튼.setFont(new Font("a옛날사진관4", Font.PLAIN, 16));
    p2공격버튼.setBounds(29, 422, 100, 42);
    panel1.add(p2공격버튼);

    p1사진.setBounds(12, 10, 245, 417);
    panel1.add(p1사진);

    p1무기가져오기.setFont(new Font("a옛날사진관4", Font.PLAIN, 16));
    p1무기가져오기.setBounds(12, 496, 129, 47);
    panel1.add(p1무기가져오기);
}
```

GUI 화면에 들어갈 개체들 기본 속성을 설정한다

```

savebutton.setFont(new Font("a옛날사진권4", Font.PLAIN, 12));
savebutton.setBounds(470, 586, 116, 46);
contentPane.add(savebutton);

savebutton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        saveToFile();
    }
});

}

protected void saveToFile() {
    PrintWriter pw = null;
    try {
        pw = new PrintWriter("c://game//result.txt");
        pw.println(textArea.getText());

        pw.flush();
        pw.close();
    } catch (FileNotFoundException e) {
        // TODO Auto-generated catch block
        System.out.println("그런 파일을 없습니다");
        e.printStackTrace();
    }
}
}

```

파일저장 버튼과 파일저장 코드 부분

```

public GameWindow(Player p1, Player p2) {

    this(); //해해주면 인자 부르지 할 수 있는거였어!!!

    p1progressBar.setMaximum(p1.getHp());
    p2progressBar.setMaximum(p2.getHp());
    p1progressBar.setValue(p1.getHp());
    p2progressBar.setValue(p2.getHp());
    p1Hp.setText(Integer.toString(p1.getHp()));
    p2Hp.setText(Integer.toString(p2.getHp()));

    p1사진.setIcon(new ImageIcon(p1.getImgfile()));
    p2사진.setIcon(new ImageIcon(p2.getImgfile()));

    startbutton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            textArea.append("<<<<<<<Game Start>>>>>>>\n");
            textArea.append("참가자 1:\n");
            showPlayer(p1);
            textArea.append("참가자 2:\n");
            showPlayer(p2);
            show_w(p1);
            show_w(p2);
        }
    });
}

```

1. 위의 생성자를 받아와 내용을 채워주는 부분.
2. startbutton을 누르면 두 참가자의 무기와 능력을 보여주며 게임을 시작한다.

```

p1공격버튼.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        p1.attack(p2);
        p1progressBar.setValue(p1.getHp());
        p2progressBar.setValue(p2.getHp());
        p1Hp.setText(Integer.toString(p1.getHp()));
        p2Hp.setText(Integer.toString(p2.getHp()));
        textArea.append("\n"+p1.name+" "+p1.cur_weapon.name+"(으)로 "+p2.name+" 공격!\n\n");

        showPlayer(p1);
        showPlayer(p2);

        end_game(p1,p2);
    }
});

```

공격버튼: 누르면 Hp bar 줄어들고, 화면 중간의 textArea에 어떤 무기로 공격했는지의 정보가 개시된다. 공격 후에는 각 플레이어의 상태를 보여줌. (p2공격버튼도 똑같음)

```

public void end_game(Player p1, Player p2) {
    if(p1.getHp() < 0 || p2.getHp() < 0) {
        System.out.println("\n=====GAME OVER=====\\n");
        textArea.append("\n=====GAME OVER=====\\n");
        if(p1.getHp() > 0) {
            textArea.append(p1.name+" Win!");
            System.out.println(p1.name+" Win!");
            panel1.setBackground(Color.GREEN);
            panel2.setBackground(Color.RED);
        }
        else {
            textArea.append(p2.name+" Win!");
            System.out.println(p2.name+" Win!");
            panel1.setBackground(Color.RED);
            panel2.setBackground(Color.GREEN);
        }
    }
}
}

```

end game 메소드는 한 플레이어의 Hp가 0이되어 게임이 끝난 것을 표현해주는 메소드. 이
 왼쪽의 판넬이 초록, 진 쪽의 판넬이 빨간색으로 변함.

```

p1무기가져오기.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        MyView2 p1무기 = new MyView2(p1, p2, textArea, p1progressBar, p2progressBar, p1Hp, p2Hp, panel1, panel2);
        p1무기.setVisible(true);
    }
});

```

플레이어가 갖고있는 무기가 나타난 창을 띄워서 직접 공격할 수 있게 도와주는 버튼.
 누르면 MyView 화면이 뜬다.

MyView

```

public MyView2(Player p1, Player p2, JTextArea a, JProgressBar p1Bar, JProgressBar p2Bar, JTextPane p1Hp, JTextPane p2Hp, JPanel panel)
{
    this();
    JButton [] weap_button = new JButton[p1.getWeapons().size()];
    for(int i=0; i<p1.getWeapons().size(); i++) {
        weap_button[i] = new JButton("button");
        weap_button[i].setText(p1.getWeapons().get(i).name);
        weap_button[i].setFont(new Font("a옛날사진관4", Font.PLAIN, 13));
        weap_button[i].setBounds(25, 62+i*75, 130, 52);
        panel.add(weap_button[i]);
    }
}

```

1. 마찬가지로 위에서 만들어놓은 기본틀을 가져와서 사용하는 생성자. 입력해주는 원소들이
 굉장히 많은데, 이는 My View에서 생기게한 변화들을 GameWindow에서 표현해야하기 때문
 이었다.
2. My view에 뜨는 버튼의 개수는 플레이어가 갖는 버튼의 개수에 맞춰서 생성된다.

1. 위에서 생겨난 버튼에 기능을 더해주는 부분. final로 상수를 설정하지 않았더니 for 문이
 버튼의 action을 넣는 코드에 들어가지 않아서 만드는 데에 고생했다.

```

for (int k=0; k < p1.getWeapons().size(); k++) {
    final int temp = k;
    weap_button[k].addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {

            p1.attact_choose(p2, p1.getWeapons().get(temp));
            a.append("\n" + p1.name + ": " + p1.getWeapons().get(temp).name + "(으)로 " + p2.name
                + " 공격!\n\n");
            //Game Window에서 Hp Bar와 text 바꾸기 위한 코드
            change HpInfo(p1,p2,p1Bar,p2Bar,p1Hp,p2Hp);

            //필 or 중은 공격 후에 남은 화살 or 총알 정보를 보여줌 (힐크는 필/중도 던지기 때문에 정보 필요없음)
            if(! (p1 instanceof 힐크) && p1.getWeapons().get(temp) instanceof 필) {
                show_arrow((필)p1.getWeapons().get(temp),p1,a);
            }
            if(! (p1 instanceof 힐크) && p1.getWeapons().get(temp) instanceof 중) {
                show_bullet(p1,(중)p1.getWeapons().get(temp),a);
            }

            showPlayer(p1, a);
            showPlayer(p2, a);

            end_game(p1, p2, a, panel1, panel2);

        }
    });
}
}

```

2. 버튼 누를 때마다 그에 맞는 공격이 이뤄질 수 있도록 하는 코드
3. GameWindow에서 설정한 bar변화 코드가 Myview에서 생기는 변화까지 반영하지 않아서 만든 change Info 함수.
4. 화살/ 총알 남은 개수를 보여주기 위해 추가한 코드

```

}
}
public void show_arrow(화 w, Player p, JTextArea a) {
    if(w.get화살()<=0) {
        a.append("남은 화살: " + w.get화살()+"\n");
        a.append(w.name+" : 화살이 다 떨어져 공격하지 못합니다!\n");
        a.append(w.name+" : 무기창고에서 제거!\n");
        p.removeWeapons(w);
        show_w(p,a);
    }
    else{
        a.append("남은 화살: " + w.get화살()+"\n");
    }
}
}
public void show_bullet(Player p,중 w, JTextArea a) {
    if(w.get총알()<=0) {
        a.append("남은 총알: " + w.get총알()+"\n");
        a.append(w.name+" : 총알이 다 떨어져 공격하지 못합니다!\n");
        a.append(w.name+" : 무기창고에서 제거!\n");
        p.removeWeapons(w);
        show_w(p,a);
    }
    else{
        a.append("남은 총알: " + w.get총알()+"\n");
    }
}
}
public void change HpInfo(Player p1, Player p2,JProgressBar p1Bar,JProgressBar p2Bar,JTextPane p1Hp,JTextPane p2Hp) {
    p1Bar.setValue(p1.getHp());
    p2Bar.setValue(p2.getHp());
    p1Hp.setText(Integer.toString(p1.getHp()));
    p2Hp.setText(Integer.toString(p2.getHp()));
}
}
}

```

화살, 총알 개수 보이는 메소드와 Game Window에서 Hp bar랑 text 변하게 하는 메소드

```

}
}
public void end_game(Player p1, Player p2, JTextArea a, JPanel panel1, JPanel panel2) {
    if(p1.getHp()<0 || p2.getHp()<0) {
        System.out.println("\n=====GAME OVER=====");
        a.append("\n=====GAME OVER=====");
        if(p1.getHp()>0) {
            a.append(p1.name+" Win!");
            System.out.println(p1.name+" Win!");
            panel1.setBackground(Color.GREEN);
            panel2.setBackground(Color.RED);
        }
        else {
            a.append(p2.name+" Win!");
            System.out.println(p2.name+" Win!");
            panel1.setBackground(Color.RED);
            panel2.setBackground(Color.GREEN);
        }
    }
}
}
}

```

My view에서 공격을 실행하다 게임이 끝났을 시에도 GameWindow에 표현이 돼야 했으므로 추가한 GameWindow에서의 endgame과 유사한 메소드

Main

```
호크아이 Hawkeye = new 호크아이("호크아이");
토르 Thor = new 토르("토르");
헐크 Hulk = new 헐크("헐크");
블랙위도우 Blackwidow = new 블랙위도우("블랙위도우");
아이언맨 Ironman = new 아이언맨("아이언맨");

총 a1 = new 총("총_공용1",3);
총 a2 = new 총("총_공용2",4);
칼 b = new 칼("칼_공용");
망치 c = new 망치("망치_공용");
활 d1 = new 활("활_공용1",3);
활 d2 = new 활("활_공용2",2);
활 d3 = new 활("활_공용3",4);

ArrayList<Weapon> garrage = new ArrayList<Weapon>();
garrage.add(a1);
garrage.add(a2);
garrage.add(b);
garrage.add(c);
garrage.add(d1);
garrage.add(d2);
garrage.add(d3);

Hawkeye.settingWeapons(garrage, 2);
Thor.settingWeapons(garrage, 2);
Hulk.settingWeapons(garrage, 2);
Blackwidow.settingWeapons(garrage, 2);
Ironman.settingWeapons(garrage, 2);
```

우선 모든 플레이어를 생성해 두었고, 공용 창고에 들어갈 무기들을 생성하여 창고에 넣어주었으며, 각 플레이어가 창고에서 무기를 가져오도록 했다.

```
Hulk.show_header();
Hulk.show();
Blackwidow.show();

Hulk.show_w();
Blackwidow.show_w();

GameWindow gamewindow = new GameWindow(Hulk,Blackwidow);
gamewindow.setVisible(true);

}
```

후에 원하는 두 플레이어를 선정하여 둘을 배틀에 참여시킴.